



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

PROGRAMOZÁSI NYELVEK ÉS FORDÍTÓPROGRAMOK
TANSZÉK

Elsőrendű logika, mint algebrai elmélet formalizálása Agdában

Témavezető:

Kaposi Ambrus

egyetemi docens

Szerző:

Szilágyi Péter

programtervező informatikus MSc

Budapest, 2025

EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

DIPLOMAMUNKA TÉMABEJELENTŐ

Hallgató adatai:

Név: Szilágyi Péter

Neptun kód: CA36OR

Képzési adatok:

Szak: programtervező informatikus, mesterképzés (MA/MSc)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Kaposi Ambrus

munkahelyének neve, tanszéke: **ELTE IK, Programozási nyelvek és Fordítóprogramok Tanszék**

munkahelyének címe: **1117, Budapest, Pázmány Péter sétány 1/C.**

beosztás és iskolai végzettsége: **egyetemi docens**

A diplomamunka címe:

Elsőrendű logika, mint algebrai elmélet formalizálása Agdában

A diplomamunka témája:

A kutatás célja az elsőrendű logika szintaxisának és szemantikájának formalizálása az Agda bizonyító rendszerben. Ezt algebrai módon tesszük meg, tehát a nyelv egy algebrai elmélet, és a szintaxis az iniciális modell. Az elsőrendű logika, mint algebrai elmélet tartalmaz egyenlőségeket, de ezek egy kivételével nem számítási szabályok, hanem csak a helyettesítési kalkulusra vonatkoznak. Az egy kivétel egy egyediség szabály, amely azt mondja, hogy tetszőleges két bizonyítás egyenlő. Így az elsőrendű logika szintaxisa kvóciensek nélkül megadható Agdában, a bizonyítások a definicionálisan irreleváns Prop fajtaban vannak. Az így megadott szintaxis inicialitását bizonyítjuk (tetszőleges modellbe pontosan egy homomorfizmus megy). Megadjuk a Tarski szemantikát, a klasszikus szemantikát, a Kripke szemantikát. Esetlegesen teljesség-bizonyítás a Kripke szemantikára (a teljesség a logika negatív fragmensére vonatkozik). A dolgozatban összefoglaljuk az elsőrendű logika szokásos megadási módjait, és összehasonlítjuk az algebrai változattal.

Budapest, 2023. 12. 13.

Tartalomjegyzék

1. Köszönetnyilvánítás	4
2. Bevezetés	5
2.1. A dolgozat célja	5
2.2. Elsőrendű logika Agdában	8
2.2.1. Logikai állítások, mint típusok	8
2.2.2. Az univerzális kvantor, mint egy függő függvény típus	9
2.2.3. Az egzisztenciális kvantor ($\exists$), mint a függő pár típus (Σ)	9
2.2.4. Logikai összekötők, mint meglévő típuskonstruktorok	9
2.2.5. Egyenlőség és elsőrendű termék	10
2.2.6. Példa bizonyítás	10
3. Kapcsolódó munkák	11
4. Logika	13
4.1. Bevezetés a logikába	13
4.1.1. Ítéletlogika (nulladrendű logika)	13
4.1.2. Predikátumlogika (elsőrendű logika)	14
4.1.3. A nulladrendű és az elsőrendű logika közötti különbség	15
4.1.4. Eldönthetőség	15
4.1.5. Klasszikus és konstruktív logika	16
5. Modellfogalom	19
5.1. Kategória	19
5.2. Termék	20
5.2.1. De Bruijn-indexek	21
5.3. Formulák	23
5.4. Bizonyítások	24

5.5.	Különböző változók kezelése	24
5.6.	Reláció- és függvényszimbólumok	25
5.6.1.	Helyettesítési szabályok	25
5.7.	Logikai összekötők	26
5.7.1.	Implikáció ($\supset$)	26
5.7.2.	Konjunkció ($\wedge$)	26
5.7.3.	Diszjunkció ($\vee$)	27
5.7.4.	Konstans igaz ($\top$)	27
5.7.5.	Konstans hamis ($\perp$)	28
5.7.6.	Kvantorok	28
6.	Szintaxis	30
6.1.	Az implementáció alapelvei	30
6.1.1.	A helyettesítések kezelése	31
6.2.	Szortok	31
6.3.	Változók	32
6.4.	Termváltozók	32
6.5.	Termek és behelyettesítések	32
6.6.	Bizonyítások kontextusa és azok behelyettesítése	33
6.7.	Bizonyítások	33
6.8.	Iniciális modell	34
7.	Modellek	38
7.1.	Szemantikai modellek	38
7.2.	Bool modell	39
7.3.	Tarski modell	40
7.4.	Family modell	42
8.	Bizonyítás különböző modellekben	45
8.1.	Egyszerűbb formula	46
8.1.1.	Informális bizonyítás	46
8.1.2.	Bizonyítás a szintaxisban	46
8.1.3.	Bizonyítás a Bool modellben	46
8.1.4.	Bizonyítás a Tarski modellben	46
8.1.5.	Bizonyítás a Family modellben	46

8.2. Összetettebb formula	46
8.2.1. Informális bizonyítás	46
8.2.2. Bizonyítás a szintaxisban	46
8.2.3. Bizonyítás a Bool modellben	46
8.2.4. Bizonyítás a Tarski modellben	46
8.2.5. Bizonyítás a Family modellben	46
9. Összegzés	47
Irodalomjegyzék	49

1. fejezet

Köszönetnyilvánítás

Ezúton szeretném megköszönni konzulensemnek, Kaposi Ambrusnak a szakmai támogatását, iránymutatását és biztatását a dolgozatom elkészítése során. Hálás vagyok a 2.620-as szoba lakóinak és barátaimnak minden segítségért, ötletért és bátorításért, amely végigkísért ezen az úton. Köszönet továbbá a menedzseremnek és munkatársaimnak is a rugalmasságért és megértésért, amely lehetővé tette számomra a tanulmányaimra való koncentrációt. Végül, de nem utolsósorban természetesen köszönet a családomnak a szeretetükért, támogatásukért és türelmükért, amely nélkül nem érhettem volna el ezt az eredményt.

2. fejezet

Bevezetés

2.1. A dolgozat célja

A dolgozat célja az elsőrendű logika formalizálása Agdában, mint mélyen beágyazott nyelv. A mély beágyazás biztosítja, hogy teljes kontrollunk legyen a logika reprezentációja felett. Többféle szemantikát meg tudunk így adni, míg a sekély beágyazás rögzít egy szemantikát, mivel közvetlenül a metanyelv konstrukcióit használja.

Egy algebrai elméletet szortok, operátorok és egyenletek határoznak meg. Egy elmélet modelljeit vagy algebrait algebrai struktúrának is szokás nevezni, ez minden szorthoz egy halmazt, az operátorokhoz függvényeket, az egyenletekhez pedig egyenlőséget rendel.

Például a félcsoport egy olyan algebrai struktúra, amely egy halmazból és a rajta értelmezett asszociatív bináris műveletből áll. Ebben az esetben az asszociativitás lesz az az egyenlőség, amelyet a félcsoportnak teljesítenie kell.

```
record Semigroup : Set1 where
  field
    C : Set
     $\_ \bullet \_ : C \rightarrow C \rightarrow C$ 
    associativity :  $(x\ y\ z : C) \rightarrow (x \bullet y) \bullet z \equiv x \bullet (y \bullet z)$ 
```

Ezek alapján a $\mathbb{N}$ és a $+$ (összeadás), vagy a $\mathbb{B}$ (Bool) és az $\wedge$ (és) félcsoportot alkotnak. Ez nem mondható el viszont a $\mathbb{N}$ és a $-$ (kivonás) párosításról, ugyanis az utóbbi műveletre nem igaz, hogy asszociatív.

Az előbb említett félcsoportokat egy-egy modellként is reprezentálhatjuk a következő módon:

```

N+ : Semigroup
N+ = record
  { C = ℕ
  ; _•_ = _+_
  ; associativity = +-assoc
  }

```

A szintaxist definiálhatjuk egy olyan modellként, amelyből minden más modellbe megy egy függvény megőrizve az operátorokat. A félcsoport szintaxisát a 0 elemű halmazzal tudjuk megadni. A 0 elemű halmazon meg tudunk adni mintaillesztéssel tetszőleges más halmazba függvényt, amely triviálisan megőrzi az összes operátort.

```

data C : Set where

I : Semigroup
I = record
  { C = C
  ; _•_ = λ x y → x
  ; associativity = λ x y z → refl
  }

```

A fő kontribúció abban rejlik, hogy az iniciális algebrát kvóciensek nélkül (induktív adattípusokkal) adjuk meg. Erre egy jó példa lehet az A feletti monoid, ami egy egyenlőséges algebrai elmélet, de meg tudjuk adni a szintaxisát egyenlőségek nélkül.

```

record Monoid {A : Set} : Set1 where
  field
    C : Set
    _•_ : C → C → C
    assoc : {x y z : C} → (x • y) • z ≡ x • (y • z)
    e : C
    idl : {x : C} → e • x ≡ x
    idr : {x : C} → x • e ≡ x
    η : A → C

```



```

data List (A : Set) : Set where
  [] : List A
  _::_ : A → List A → List A

_++_ : ∀ {A} → List A → List A → List A
[] ++ ys = ys
(x :: xs) ++ ys = x :: (xs ++ ys)

assoc : ∀ {A} {x y z : List A} → (x ++ y) ++ z ≡ x ++ (y ++ z)
assoc {A} {} {} {} = refl
assoc {A} {x :: x₁} {y} {z} = cong (x :: _) (assoc {A} {x₁} {y} {z})

idr : ∀ {A} {x : List A} → x ++ [] ≡ x
idr {A} {} {} = refl
idr {A} {x :: x₁} = cong (x :: _) (idr {A} {x₁})

l : {A : Set} → Monoid {A}
l {A} = record
  { C = List A
  ; _•_ = _++_
  ; assoc = λ {x} {y} {z} → assoc {A} {x} {y} {z}
  ; e = []
  ; idl = refl
  ; idr = idr
  ; η = _:: []
  }

```

A monoid szintaxisaként itt a listát használjuk, ugyanis a listák és a konkatenáció természetes módon kielégítik a monoid egyenlőségeit (asszociativitás, identitáselem).

A következő konstrukció ellenpéldaként szolgál, nevezhetjük rendezetlen párnak:

```

record UP {A : Set} : Set₁ where
  field
    C : Set
    _',_ : A → A → C
    eq : {x y : A} → x , y ≡ y , x

```

Ahhoz, hogy megadjuk a rendezetlen pár szintaxisát, egy kiegészítő egyenletre van szükség, amely biztosítja a kommutativitást $(a, b) \equiv (b, a)$. Ebben az esetben az iniciális algebra nem adható meg egyenlőségek nélkül, mert ha megbóbálnánk az előbbi példa alapján `data`-val megadni, az `eq`-t már nem tudnánk, az Agda nem fogadná el.

Míg bizonyos algebrai struktúrák iniciális modelljei definiálhatók egyenlőségek megadása nélkül, algebrai adattípusokkal és rekurzív függvények megadásával, másokhoz további egyenletek szükségesek a kívánt tulajdonságok kikényszerítésére.

Ebben a dolgozatban az elsőrendű logika egy általánosított algebrai elméletként (GAT) lett megvalósítva, így a modell fogalmat többszortú módon adjuk, amelyek indexelve vannak egymás felett. A szortok (fajták) a következők lesznek: `Tm`, `For`, `Pf`. Így reperezántálva a kifejezéseket, formulákat, és bizonyításokat. A változók miatt pedig a `Con` és `Sub` szortokat vezetjük be, ábrázolva a környezeteket és a közöttük való átjárást.

2.2. Elsőrendű logika Agdában

Az Agda egy függően típusozott funkcionális nyelv, amelyet bizonyításokra is tudunk használni, ezzel megmutatva a logika kapcsolatát a típuselmélettel. Továbbá egy teljes nyelv is, ami azt jelenti, hogy minden program terminál és mintaillesztés során is minden esetet kötelező lekezelni.

Ahogy már korábban szó volt róla, az elsőrendű logikát mint nyelvet sekély módon is be lehet ágyazni az Agda rendszerébe, mivel ehhez minden eszköz adott a meglévő típusok és műveletek által.

Az elsőrendű logika sekély beágyazásának megvalósítása Agdában meglehetősen könnyedén elérhető.

2.2.1. Logikai állítások, mint típusok

Az elsőrendű logikában egy formula, mint $P(x)$, vagy igaz, vagy hamis. Agdában kezelhetjük az állításokat típusokként, így a bizonyítása a $P(x)$ formulának megfeleltethető az állítás szort egy elemének.

2.2.2. Az univerzális kvantor, mint egy függő függvény típus

Az univerzális kvantor $\forall x.P(x)$ egyszerűen reprezentálható egy függő függvény típussal:

```

 $\forall\_ : \{A : \text{Set}\} \rightarrow (A \rightarrow \text{Prop}) \rightarrow \text{Prop}$ 
 $\forall P = \forall x \rightarrow P\ x$ 

```

2.2.3. Az egzisztenciális kvantor ($\exists$), mint a függő pár típus (Σ)

Az egzisztenciális kvantor $\exists x.P(x)$ megfeleltethető a függő pár típusnak:

```

data  $\exists$  (A : Set) (B : A  $\rightarrow$  Prop) : Prop where
   $\_ , \_ : (a : A) \rightarrow B\ a \rightarrow \exists\ A\ B$ 

```

2.2.4. Logikai összekötők, mint meglévő típuskonstruktorok

- Konjunkció ($\wedge$) - szorzat típus (Σ -types):

```

record  $\_ \Sigma \_$  (A B : Set) : Set where
  constructor  $\langle \_, \_ \rangle$ 
  field
    fst : A
    snd : B

```

- Disjunkció ($\vee$) - összeg típus:

```

data  $\_ \uplus \_$  (A B : Set) : Set where
  inl : A  $\rightarrow$  A  $\uplus$  B
  inr : B  $\rightarrow$  A  $\uplus$  B

```

- Implikáció ($\rightarrow$) - függvény típus:

```

 $\_ \rightarrow \_ : \text{Set} \rightarrow \text{Set} \rightarrow \text{Set}$ 
A  $\rightarrow$  B = A  $\rightarrow$  B -- Az Agda függvény típusa

```

- **Negáció** ($\neg A$) - $A \rightarrow \perp$ (ahol $\perp$ az üres típus):

$$\neg_ : \text{Set} \rightarrow \text{Set}$$

$$\neg A = A \rightarrow \perp$$

2.2.5. Egyenlőség és elsőrendű termek

A beépített egyenlőség Agdában ($_ \equiv _$), amelyet közvetlenül használhatunk:

postulate

$$_ \equiv _ : \{A : \text{Set}\} \rightarrow A \rightarrow A \rightarrow \text{Set}$$

Mindezekkel a funkciókkal a sekély beágyazás egyszerűen megvalósítható. Az elsőrendű logikát nem szükséges szintaktikus fákként kódolni, hanem egyszerűen a beépített Agda típusokat és függvényeket használhatjuk.

2.2.6. Példa bizonyítás

Például, ha be akarnánk bizonyítani a következőt:

$$\forall x. (P(x) \rightarrow Q(x)) \rightarrow (\forall x. P(x)) \rightarrow (\forall x. Q(x))$$

Egyszerűen csak egy függvény applikációval adjuk meg:

$$\text{lemma} : \{A : \text{Set}\} \{x : A\} \{P Q : A \rightarrow \text{Set}\} \rightarrow$$

$$(\forall x \rightarrow P x \rightarrow Q x) \rightarrow (\forall x \rightarrow P x) \rightarrow \forall x \rightarrow Q x$$

$$\text{lemma } f \ g \ x = f \ x \ (g \ x)$$

3. fejezet

Kapcsolódó munkák

Ebben a fejezetben összefoglaljuk azokat a korábbi kutatásokat és formalizációkat, amelyek közvetlenül kapcsolódnak az elsőrendű logika algebrai szemléletű megközelítéséhez és Agdában történő formalizálásához. Áttekintjük a témához szorosan kapcsolódó dolgozatokat, beszámolókat és egyéb formális rendszerekben végzett hasonló munkákat.

A kapcsolódó munkák között érdemes megemlíteni Fenyvesi Róbert MSc disszertációját [1], amely a nulladrendű klasszikus logika különböző megadásainak ekvivalenciáját vizsgálja algebrai úton, az Agda segítségével. Munkája közeli rokonságot mutat jelen dolgozat céljaival, különösen az algebrai szemlélet miatt, azonban a nulladrendű logikával foglalkozik több oldalról, míg a jelen dolgozat az elsőrendű logikát célozza meg.

Samy Avrillon beszámolója [2], amely az ELTE-n töltött gyakornoki kutatási eredményeit foglalja össze, jelen dolgozat elődjének tekinthető, ugyanis a célkitűzések és az alkalmazott módszerek nagyban hasonlóak. A munkájában részletesebben tárgyalja az elsőrendű logika formalizálásának módszertani alapjait, különös tekintettel a másodrendű általánosított algebrai elméletek (SOGAT) általánosított algebrai elméletekké (GAT) történő átírására támaszkodva. A diplomamunka célja a megközelítés továbbfejlesztése, a formalizálás teljessé tétele, többféle modell (Bool, Tarski, Family) definiálása, valamint az iniciális modell és a szemantikák részletes kidolgozása, ezek összehasonlítása és további tulajdonságok vizsgálata.

Altenkirch, Hofmann és Streicher klasszikus munkája [3] a nulladrendű intuicionista logika teljességének bizonyítására koncentrál, kategóriaelméleti eszközökkel.

Az algebrai szemlélet elméleti háttéréül szolgál többek között Awodey és Bauer

bevezetője a kategóriaelméleti logikába [4]. Bár ez a megközelítés nem gépileg formalizált, a logika algebrai megközelítéséhez szoros kapcsolat fűzi. Átfogó képet nyújt az algebrai struktúrák és a logikai rendszerek kapcsolatáról.

A témához szorosan kapcsolódik több Coq-ban végzett formalizálás is, amelyek közül az egyik [5] az elsőrendű logika formális leírására és annak metateóriájára koncentrálnak, elsősorban gyakorlati, típuselméleti eszközöket alkalmazva. Bár nem algebrai megközelítést követ, a formalizálási technikák hasonlóságai miatt hasznos összehasonlítási alapot kínál.

Egy másik kapcsolódó munka [6] a nulladrendű logika teljességi tételének formalizált bizonyítását nyújtja Coq-ban. Ez a munka különösen fontos a jelen dolgozat szempontjából, mivel ugyanarra a logikai fragmentumra koncentrálnak, mint a dolgozat algebrai irányból történő megközelítése, ám eltérő eszközökkel dolgozik.

4. fejezet

Logika

Ez a fejezet a logika alapfogalmait ismerteti, különös tekintettel a nulladrendű és elsőrendű logikára. Bemutatjuk az alapvető logikai műveleteket, a kvantorokat, valamint a klasszikus és konstruktív logika közötti különbségeket, amelyek ismerete szükséges a későbbi formalizáláshoz.

4.1. Bevezetés a logikába

A logika a helyes következtetés és érvelés tudománya. Matematikai szempontból a logika egy olyan formális nyelv vagy rendszer, amely lehetővé teszi az állítások precíz leírását, valamint ezek igazságértékének és egymáshoz való viszonyának vizsgálatát. A formális logika célja, hogy matematikailag megragadható módon írjon le következtetéseket, amelyek objektív, jól meghatározott szabályokon alapulnak.

A különböző típusú logikák eltérő kifejezőerővel rendelkeznek. A legegyszerűbb formális logikai rendszer a nulladrendű logika, más néven propozíciós logika, vagy ítéletlogika, amelyet a predikátumlogika vagy elsőrendű logika követ, amely már képes az egyedekekről és azok tulajdonságairól is beszélni.

4.1.1. Ítéletlogika (nulladrendű logika)

Az ítéletlogika olyan állításokkal dolgozik, amelyek teljes egészükben igazak vagy hamisak, és nem bonthatók tovább kisebb részekre anélkül, hogy elveszítenék jelentésüket. Ezeket az állításokat propozícióknak nevezzük, és általában kisbetűkkel jelöljük, például p , q , r .

A propozíciókhoz logikai műveletek tartoznak, mint például:

- Negáció ($\neg p$) – "nem p"
- Konjunkció ($p \wedge q$) – "p és q"
- Diszjunkció ($p \vee q$) – "p vagy q"
- Implikáció ($p \rightarrow q$) – "ha p, akkor q"
- Ekvivalencia ($p \leftrightarrow q$) – "p akkor és csak akkor, ha q"

A ítéletlogika célja annak eldöntése, hogy bizonyos formulák egy adott logikai rendszerben igazak-e, illetve hogy egyes állítások logikailag következnek-e másokból.

4.1.2. Predikátumlogika (elsőrendű logika)

Az elsőrendű logika tovább lép a nulladrendű logika korlátain. Itt már nem csupán állításokat vizsgálunk, hanem azok belső szerkezetét is feltárjuk. A predikátumlogika képes kifejezni olyan állításokat, mint:

- "Minden ember halandó"
- "Van olyan szám, amely páros"

Ez úgy éri el, hogy egyedekre (individuumokra), tulajdonságokra (predikátumokra) és kapcsolatokra épít. A rendszer alapelemei:

- Változók (pl. 'x', 'y', 'z')
- Konstansok (pl. 'a', 'b', '0', '1')
- Függvényszimbólumok (pl. 'f(x)')
- Predikátumszimbólumok (pl. 'Ember(x)', 'Szeret(x, y)')
- Kvantorok:
 - Univerzális kvantor ($\forall x$) – "minden x-re"
 - Egzisztenciális kvantor ($\exists x$) – "létezik olyan x"

Például a mondat:

Minden ember halandó

formalizálva így néz ki:

$$\forall x. \text{Ember}(x) \rightarrow \text{Halandó}(x)$$

4.1.3. A nulladrendű és az elsőrendű logika közötti különbség

A két logikai rendszer között a fő különbség az elemzési mélységben és kifejezőerőben rejlik:

Tulajdonság	Nulladrendű logika	Elsőrendű logika
Alapegység	Egész állítások (pl. "esik az eső")	Egyedek, tulajdonságok és relációk
Belső szerkezet	Nem vizsgálja	Vizsgálja és kezeli
Kvantorok	Nincsenek	Vannak ($\forall$, $\exists$)
Kifejezőerő	Korlátozott	Sokkal gazdagabb
Példa	$p \rightarrow q$	$\forall x. \text{Ember}(x) \rightarrow \text{Halandó}(x)$

4.1. táblázat. A nulladrendű és az elsőrendű logika közötti különbségek.

Az ítéletlogika tehát alkalmas egyszerű állítások kezelésére, míg az elsőrendű logika képes összetettebb szerkezeteket és általános állításokat is kezelni. Ennek következtében az elsőrendű logika az automatikus bizonyítás, formális verifikáció és matematikai elméletek formalizálásának is egyik alapja. Például Agdában is ezen alapul a legtöbb formális modell.

4.1.4. Eldönthetőség

A logikai rendszerek egyik központi kérdése az eldönthetőség. Egy logikai rendszer eldönthető, ha létezik olyan algoritmus, amely minden lehetséges bemenetre, véges időn belül eldönti, hogy az adott formula érvényes vagy nem, vagy hogy egy adott formula levezethető-e a rendszer axiómáiból és szabályaiból.

Ez különösen fontos a formális verifikációban és az automatikus tételbizonyításban, ahol azt szeretnénk, hogy egy gép automatikusan meg tudja mondani, hogy egy adott logikai állítás igaz-e vagy sem.

Eldönthetőség a nulladrendű logikában

A nulladrendű logika eldönthető: létezik algoritmus, amely képes eldönteni bármely propozíciós formula esetén, hogy az logikailag érvényes-e. Az egyik ilyen algoritmus például az igazságtábla-módszer, amely minden lehetséges értékkombinációt kipróbál.

Vegyünk például az alábbi formulát:

$$(p \wedge (p \rightarrow q)) \rightarrow q$$

Ez egy tautológia, azaz mindig igaz, és ezt géppel is eldönthetjük véges időn belül.

Eldönthetőség az elsőrendű logikában

Az elsőrendű logika általánosságban nem eldönthető. Ez azt jelenti, hogy nem létezik olyan algoritmus, amely minden elsőrendű formula esetén képes lenne megmondani véges időn belül, hogy az levezethető-e vagy sem.

Ez a híres Church–Turing-tétel egyik következménye, amely kimondja, hogy az elsőrendű logika bizonyíthatósága rekurzívan enumerálható, de nem rekurzívan eldönthető.

Vannak azonban esetek, ahol eldönthetőség biztosítható, ha valamilyen formában korlátokat vezetünk be, például formulakészletek szűkítése, vagy véges struktúrákra korlátozott értelmezések esetén.

Az eldönthetőség hiánya azt jelenti, hogy az elsőrendű logikában nem mindig lehet automatizálni a bizonyítást. Egy automatikus tételbizonyító rendszer, mint például a Z3 [7], nem garantálhatja, hogy minden helyes állítást képes lesz levezetni. Az ilyen rendszerek gyakran fél-automaták: bizonyos esetekben képesek segíteni vagy teljesen elvégezni a bizonyítást, de általános megoldás nem létezik.

4.1.5. Klasszikus és konstruktív logika

A klasszikus logika a legelterjedtebb, amelyet a legtöbb hagyományos matematikai elmélet is használ. Ezzel szemben a konstruktív logika – más néven intuicionista logika – eltér bizonyos alapvető logikai szabályok elfogadásában, különösen azokra az esetekre vonatkozóan, amikor valaminek a létezését vagy valami igazságát állítjuk anélkül, hogy konkrét példát tudnánk adni vagy bizonyítékot tudnánk konstruálni.

A klasszikus logika főbb jellemzői

A klasszikus logika elfogadja az alábbi törvényeket:

- Kizárt harmadik elve: $P \vee \neg P$ minden P formula esetén igaz.
- Kettős negáció elve: $\neg\neg P \rightarrow P$ minden P formula esetén igaz.

- Nem-konstruktív létezés bizonyítása: elég kimutatni, hogy a létezés tagadása ellentmondáshoz vezet.

Ezek az elvek lehetővé teszik sok klasszikus matematikai bizonyításfajta alkalmazását, például:

- Indirekt bizonyítás (reductio ad absurdum)
- Nem konstruktív létezés bizonyítása (pl. "létezik irracionális szám ' a ', amire a^a racionális" – de nem mondjuk meg, melyik az)

A konstruktív logika főbb jellemzői

A konstruktív logika nem fogadja el automatikusan a kizárt harmadik elvét, sem a kettős negáció elvét. Ehelyett csak akkor állíthatjuk, hogy egy állítás igaz:

- ha konkrét bizonyítékot tudunk adni rá (pl. konstrukciót vagy algoritmust),
- vagy ha például $\exists x. P(x)$ állítást vizsgáljuk, akkor meg kell mutatni konkrétan az ' x '-et, amire $P(x)$ igaz.

Ez a logika különösen jól illeszkedik a kategóriaelmélethez, típuselmélethez és a formális bizonyítási rendszerekhez. Ezekben a rendszerekben egy állítás bizonyítása gyakran azt jelenti, hogy megadunk egy konkrét értéket vagy programot, amely megfelel az állításnak.

Állítás	Klasszikus logika szerint	Konstruktív logika szerint
$P \vee \neg P$	Mindig igaz	Nem mindig igaz (bizonyíték kell valamelyik ágra)
$\neg\neg P \rightarrow P$	Igaz	Nem elfogadott általánosan
$\exists x. P(x)$	Elég indirekt bizonyítás	Csak konkrét x bemutatása mellett igaz

4.2. táblázat. Különbségek a klasszikus és konstruktív logika között.

Kapcsolat az Agdával

Az Agda és más hasonló rendszerek (pl. Coq, Lean) konstruktív logikát alkalmaznak: amikor egy állítást bebizonyítunk Agdában, valójában egy értéket konstruálunk, amely az adott állítás típusának felel meg. Ennek alapja a Curry–Howard megfelelés, amely szerint:

- Állítások $\leftrightarrow$ Típusok
- Bizonyítások $\leftrightarrow$ Programok (értékek)

Ez a szemlélet a konstruktív logikához illeszkedik, hiszen minden állításnak csak akkor van "értelme", ha tudunk adni rá konkrét példát, vagy meg tudjuk konstruálni a bizonyítását.

5. fejezet

Modellfogalom

Ez a fejezet bemutatja az algebrai adattípusként megadott elsőrendű logika modell fogalmát, részletes leírást adva az elemeiről és azok működéséről, valamint magyarázatot ad a munka során hozott döntések háttéréről. Az iniciális modell részleteit a következő fejezetben mutatjuk be.

5.1. Kategória

A modellfogalom első elemeként megadunk egy kontextusok és behelyettesítések által alkotott kategóriát. A kategória objektumait a kontextusok reprezentálják és a **Con** szort határozza meg. A morfizmusai pedig a behelyettesítések lesznek, amelyeket a **Sub** szort ad meg.

$$\mathbf{Con} : \mathbf{Set} \, i$$

$$\mathbf{Sub} : \mathbf{Con} \rightarrow \mathbf{Con} \rightarrow \mathbf{Set} \, (k \sqcup l)$$

A kategóriákban a morfizmusoknak kell rendelkezniük egy kompozíciós művelettel ($_ \circ _$) is, amely két morfizmus összekapcsolását végzi el, ezzel létrehozva egy új morfizmust.

$$_ \circ _ : \forall \{\Gamma \, \Delta \, \Theta\} \rightarrow \mathbf{Sub} \, \Delta \, \Gamma \rightarrow \mathbf{Sub} \, \Theta \, \Delta \rightarrow \mathbf{Sub} \, \Theta \, \Gamma$$

$$\begin{aligned} \mathbf{ass} : \forall \{\Gamma \, \Delta \, \Theta \, \Xi\} \{ \gamma : \mathbf{Sub} \, \Delta \, \Gamma \} \{ \delta : \mathbf{Sub} \, \Theta \, \Delta \} \{ \theta : \mathbf{Sub} \, \Xi \, \Theta \} \\ \rightarrow (\gamma \circ \delta) \circ \theta \equiv \gamma \circ (\delta \circ \theta) \end{aligned}$$

Tehát, ha van egy morfizmus a Δ és Γ kontextusok között, valamint egy másik morfizmus a Θ és Δ között, akkor a kompozíciójuk egy új morfizmust ad a Θ és

Γ kontextusok között. A kompozíciónak továbbá asszociatívnak kell lennie, amit az **ass** fejez ki.

A kategóriában minden objektumhoz léteznie kell egy olyan morfizmusnak, amely az objektumot önmagába képezi, ez lesz az identitás. Az identitás morfizmusnak a kompozícióval való alkalmazása nem változtathatja meg a kompozícióban szereplő másik morfizmust. Ezt az **idl** és **idr** egyenlőség biztosítja.

$$\begin{aligned} \text{id} &: \forall \{\Gamma\} \rightarrow \text{Sub } \Gamma \ \Gamma \\ \text{idl} &: \forall \{\Gamma \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow \text{id} \circ \gamma \equiv \gamma \\ \text{idr} &: \forall \{\Gamma \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow \gamma \circ \text{id} \equiv \gamma \end{aligned}$$

A terminális objektum egy olyan objektum, amelyhez minden más objektumból létezik pontosan egy morfizmus. A mi esetünkben ez a terminális objektum a $\diamond$, a hozzá tartozó morfizmus pedig az ε . A morfizmus egyedülállóságát a $\diamond\eta$ fogalmazza meg.

$$\begin{aligned} \diamond &: \text{Con} \\ \varepsilon &: \forall \{\Gamma\} \rightarrow \text{Sub } \Gamma \ \diamond \\ \diamond\eta &: \forall \{\Gamma\} \{\sigma : \text{Sub } \Gamma \ \diamond\} \rightarrow \sigma \equiv \varepsilon \end{aligned}$$

5.2. Termek

A termeket (**Tm**) egy funktorként adjuk meg. Úgy is mondhatjuk, hogy egy kontextustól függenek, mivel lehetnek benne termváltozók, amikbe be tudunk majd helyettesíteni, így értéket adva a szabad változóknak. Ehhez szükségünk lesz termekben való behelyettesítésre, másnéven egy morfizmusok feletti akcióra. Egy funktornak teljesítenie kell két feltételt is: a kompozícióval való kompatibilitást és az identitással való kompatibilitást. Ezeket az $[\circ]^t$ és $[\text{id}]^t$ egyenlőségek biztosítják.

$$\begin{aligned} \text{Tm} &: \text{Con} \rightarrow \text{Set } k \\ _[_]^t &: \forall \{\Gamma \ \Delta\} \rightarrow \text{Tm } \Gamma \rightarrow \text{Sub } \Delta \ \Gamma \rightarrow \text{Tm } \Delta \\ [\circ]^t &: \forall \{\Gamma \ \Delta \ \theta\} \{t : \text{Tm } \Gamma\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \{\delta : \text{Sub } \theta \ \Delta\} \\ &\rightarrow t \ [\ \gamma \circ \delta \]^t \equiv t \ [\ \gamma \]^t \ [\ \delta \]^t \\ [\text{id}]^t &: \forall \{\Gamma\} \{t : \text{Tm } \Gamma\} \rightarrow t \ [\ \text{id} \]^t \equiv t \end{aligned}$$

A környezetek termekkel és termváltozókkal való kiegészítését is megadjuk. A környezetbe betenni egy termváltozót ($_ \vdash_t$) hasonlítható egy olyan Descartes szorzat-

hoz, amelynek az egyik oldalán az egyelemű halmaz található: $_ \times \top$. A behelyettesítéseket is kiegészíthetjük termekkel $(_, t _)$, amelyeket ismételten a Descartes szorzattal lehetne azonosítani. A képzett pároknak tehát meg tudjuk adni az első (p_t) és a második (q_t) projekcióját.

$$\begin{aligned} _ \triangleright_t &: \text{Con} \rightarrow \text{Con} \\ _, t _ &: \forall \{\Gamma \Delta\} \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{Tm } \Delta \rightarrow \text{Sub } \Delta (\Gamma \triangleright_t) \\ p_t &: \forall \{\Gamma\} \rightarrow \text{Sub } (\Gamma \triangleright_t) \Gamma \\ q_t &: \forall \{\Gamma\} \rightarrow \text{Tm } (\Gamma \triangleright_t) \end{aligned}$$

Fontos megadjuk a redukciós szabályokat is (β, η) , hogy biztosítsuk az azonos kifejezések közötti egyenlőséget.

$$((p \circ _) , (q[_])) : \text{Sub } \Delta (\Gamma \triangleright_t) \cong \text{Sub } \Delta \Gamma \times \text{Tm } \Delta : _, t _$$

- β : jobbról balra, majd jobbra = mintha nem történt volna semmi
- η : balról jobbra, majd balra = mintha nem történt volna semmi

$$\begin{aligned} \triangleright_t \beta_1 &: \forall \{\Gamma \Delta\} \{t : \text{Tm } \Delta\} \{\gamma : \text{Sub } \Delta \Gamma\} \rightarrow p_t \circ (\gamma _, t t) \equiv \gamma \\ \triangleright_t \beta_2 &: \forall \{\Gamma \Delta\} \{t : \text{Tm } \Delta\} \{\gamma : \text{Sub } \Delta \Gamma\} \rightarrow q_t [\gamma _, t t]^t \equiv t \\ \triangleright_t \eta &: \forall \{\Gamma \Delta\} \rightarrow \{\gamma t : \text{Sub } \Delta (\Gamma \triangleright_t)\} \rightarrow \gamma t \equiv (p_t \circ \gamma t _, t q_t [\gamma t]^t) \end{aligned}$$

5.2.1. De Bruijn-indexek

A De Bruijn-indexek a változók lambda-kalkulus vagy logikai kifejezésekben való ábrázolásának egy módja. Fő előnye, hogy nem használ változóneveket. Ahelyett, hogy egy változóra névvel hivatkoznánk, minden változót egy számmal reprezentálunk, amely megadja, hogy hány kötőelemet (például kvantorokat) kell felfelé haladnunk a kifejezésfában, hogy megtaláljuk a változót kötő operátort. A De Bruijn-indexek használata megkönnyíti a kifejezések manipulálását és egyszerűsíti a bizonyításokat, mivel nem fordul elő névütközés vagy változó-újraderfinálás.

Egy egyszerű példa:

A $\lambda x. \lambda y. x$ kifejezés De Bruijn-indexekkel: $\lambda. \lambda. 1$.

Az 1 itt a két szinttel feljebb kötött változóra hivatkozik (0-tól indexelve), azaz az x-re.

Az indexelés a következőképp működik:

- A legbelső kötött változó a 0 indexet kapja.
- Az azt követő az 1 indexet.
- Aztán 2 és így tovább.

Tehát:

- A $\lambda x. x$ kifejezés $\lambda. 0$ lesz.
- A $\lambda x. \lambda y. x$ kifejezés $\lambda. \lambda. 1$ lesz.
- A $\lambda x. \lambda y. y$ kifejezés $\lambda. \lambda. 0$ lesz.

Miért "jobbak" a De Bruijn-indexek?

Különösen az olyan formális rendszerekben, mint jelen esetben az elsőrendű logikai implementáció, a De Bruijn-indexek segítenek a következőkben:

- A változónevek ütközésének elkerülése

Egy helyettesítés végrehajtásakor az átnevezés (renaming, alfa konverzió) szükségtelenné válik. Nem kell többé aggódni a változók véletlen elfogása (variable capture) miatt.

- Helyettesítések egyszerűbb végrehajtása

A helyettesítés következetessé válik. Nincs szükség a változók nevének nyomon követésére, csak az indexeket kell megfelelően eltolni a helyettesítés során.

- Egyszerűbb egyenlőségellenőrzés

Két kifejezés az átnevezésig egyenlő, ha De Bruijn-ábrázolásuk megegyeznek. Mondhatni ingyen kapjuk az alfa-ekvivalenciát.

- Jobban illik a mechanikus bizonyítási rendszerekhez Az olyan rendszerek, mint az Agda, a Coq vagy a logikát belsőleg implementáló bizonyítási asszisztensek előnyben részesülnek a De Bruijn-indexek használatával kapcsolatban, mert:

- Függő típusos nyelvekben könnyebb őket megvalósítani
- A mintaillesztés is kevésbé érzékeny a hibákra

- Passzol a strukturális rekurziós elvekhez az olyan totális nyelvekben, mint az Agda.

Az univerzális kvantor szignatúrája tipikusan a következőképpen nézne ki:

$\forall_ : \text{Formula} \rightarrow \text{Formula}$

De szemantikusan, a De Bruijn-indexelt környezetben, a $\forall$ köt egy változót a formulában, ami azt eredményezi, hogy egyel kevesebb szabad változó lesz benne, ami a szignatúrában is látszik:

$\forall : (\mathbf{A} : \text{For } (\Gamma \triangleright_t)) \rightarrow \text{For } \Gamma$

Összességében esetünkben nagyban megkönnyíti a munkát a

- $\forall$ és $\exists$ implementálásánál és használatánál
- behelyettesítésekkel kapcsolatos műveleteknél
- formális bizonyításoknál

Jelen implementációban a De Bruijn-indexek megléte a következőképpen valósul meg:

- A $_ \triangleright_t$ művelet azáltal ad hozzá egy új változót a környezethez, hogy az indexeket eltolja egyel.
- A $_ , t _$ megfeltethető azzal, hogy az új változóhoz konstruálunk egy behelyettesítést.
- A p_t és q_t pedig a projekciók, amelyek segítenek lebontani a behelyettesítést.
 - p_t eldobja a legújabb változót
 - q_t a változó a 0. indexen

Ugyanez a megvalósítás látható a későbbiekben a bizonyítások esetében is.

5.3. Formulák

A formulákra is tekinthetünk funktorként, így hasonlóan a termékhez a következő módon adjuk meg:

$\text{For} : \text{Con} \rightarrow \text{Set } j$

$_ [_]^f : \forall \{\Gamma \Delta\} \rightarrow \text{For } \Gamma \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{For } \Delta$

$$\begin{aligned}
 [\circ]^f &: \forall \{\Gamma \Delta \theta\} \{A : \text{For } \Gamma\} \{\gamma : \text{Sub } \Delta \Gamma\} \{\delta : \text{Sub } \theta \Delta\} \\
 &\rightarrow A [\gamma \circ \delta]^f \equiv A [\gamma]^f [\delta]^f \\
 [\text{id}]^f &: \forall \{\Gamma\} \{A : \text{For } \Gamma\} \rightarrow A [\text{id}]^f \equiv A
 \end{aligned}$$

5.4. Bizonyítások

A bizonyításokat már egy kicsivel bonyolultabban adjuk meg, ugyanis egy **For** feletti függő funktornak is nevezhetnénk. Ha vesszük például a $\Gamma \vdash A$ szokásos logikai (bizonyításelméleti) jelölést, akkor a $\mathbf{p} : \mathbf{Pf} \Gamma A$ azt jelenti, hogy $\mathbf{p}$ bizonyítja az A állítást, ahol a szabad változók Γ -ban vannak.

$$\begin{aligned}
 \mathbf{Pf} &: (\Gamma : \mathbf{Con}) \rightarrow \text{For } \Gamma \rightarrow \mathbf{Prop} \, l \\
 []^\mathbf{p} &: \forall \{\Gamma \Delta A\} \rightarrow \mathbf{Pf} \Gamma A \rightarrow (\gamma : \text{Sub } \Delta \Gamma) \rightarrow \mathbf{Pf} \Delta (A [\gamma]^\mathbf{p})
 \end{aligned}$$

A bizonyítás-változókat a termváltozók esetében is használt operációkhoz hasonlóan valósítjuk meg:

$$\begin{aligned}
 _ \triangleright_{\mathbf{p}} _ &: (\Gamma : \mathbf{Con}) \rightarrow \text{For } \Gamma \rightarrow \mathbf{Con} \\
 _ \cdot_{\mathbf{p}} _ &: \forall \{\Delta \Gamma A\} \rightarrow (\gamma : \text{Sub } \Delta \Gamma) \rightarrow \mathbf{Pf} \Delta (A [\gamma]^\mathbf{p}) \rightarrow \text{Sub } \Delta (\Gamma \triangleright_{\mathbf{p}} A) \\
 \mathbf{p}_{\mathbf{p}} &: \forall \{\Gamma A\} \rightarrow \text{Sub } (\Gamma \triangleright_{\mathbf{p}} A) \Gamma \\
 \mathbf{q}_{\mathbf{p}} &: \forall \{\Gamma A\} \rightarrow \mathbf{Pf} (\Gamma \triangleright_{\mathbf{p}} A) ((A [\mathbf{p}_{\mathbf{p}}]^\mathbf{p})) \\
 \triangleright_{\mathbf{p}} \beta_1 &: \forall \{\Gamma \Delta A\} \{\gamma : \text{Sub } \Delta \Gamma\} \{a : \mathbf{Pf} \Delta (A [\gamma]^\mathbf{p})\} \rightarrow \mathbf{p}_{\mathbf{p}} \circ (\gamma \cdot_{\mathbf{p}} a) \equiv \gamma \\
 \triangleright_{\mathbf{p}} \eta &: \forall \{\Gamma \Delta A\} \{\gamma a : \text{Sub } \Delta (\Gamma \triangleright_{\mathbf{p}} A)\} \\
 &\rightarrow \gamma a \equiv (\mathbf{p}_{\mathbf{p}} \circ \gamma a \cdot_{\mathbf{p}} \text{substP } (\mathbf{Pf} \Delta) ([\circ]^{f-1}) (\mathbf{q}_{\mathbf{p}} [\gamma a]^\mathbf{p}))
 \end{aligned}$$

fix this

$$((\mathbf{p} \circ _), (\mathbf{q} [_]^\mathbf{p})) : \text{Sub } \Delta (\Gamma \triangleright_{\mathbf{p}} A) \cong (\gamma : \text{Sub } \Delta \Gamma) \times \mathbf{Pf} \Delta (A [\gamma]^\mathbf{p}) :$$

$_ \triangleright_{\mathbf{p}} _$

5.5. Különböző változók kezelése

Mint láthattuk a modellfogalomban a egységesen kezeljük a term- és a bizonyítás-változókat. Ezt a későbbiekben a szintaxisban ketté fogjuk bontani és egy külön adjuk meg a term- és bizonyítás-kontextust valamint az ezekhez tartozó helyettesítéseket.

5.6. Reláció- és függvényszimbólumok

A modell a függvények és a relációk arításával van felparaméterezve, ezt felhasználva határozzuk meg, hogy egy reláció vagy függvény hány termet vár majd paraméterként.

$$\begin{aligned}
\text{Rel} &: \forall \{\Gamma\} \{n : \mathbb{N}\} \rightarrow \text{relar } n \rightarrow \text{Tm } \Gamma \wedge n \rightarrow \text{For } \Gamma \\
\text{Rel}[] &: \forall \{\Gamma\} \{n\} \{ar : \text{relar } n\} \{ts : \text{Tm } \Gamma \wedge n\} \{\Delta\} \{\gamma : \text{Sub } \Delta \Gamma\} \\
&\rightarrow \text{Rel } ar \ ts \ [\ \gamma \]^f \equiv \text{Rel } ar \ (\text{map } _ [\ \gamma \]^t \ ts) \\
\text{fun} &: \forall \{\Gamma\} \{n : \mathbb{N}\} \rightarrow \text{funar } n \rightarrow \text{Tm } \Gamma \wedge n \rightarrow \text{Tm } \Gamma \\
\text{fun}[] &: \forall \{\Gamma\} \{n\} \{ar : \text{funar } n\} \{ts : \text{Tm } \Gamma \wedge n\} \{\Delta\} \{\gamma : \text{Sub } \Delta \Gamma\} \\
&\rightarrow \text{fun } ar \ ts \ [\ \gamma \]^t \equiv \text{fun } ar \ (\text{map } _ [\ \gamma \]^t \ ts)
\end{aligned}$$

Láthatóan itt megjelennek már a helyettesítési szabályok is. A $\text{Rel}[]$ szabály a relációk helyettesítését, míg a $\text{fun}[]$ a függvények helyettesítését végzi el.

5.6.1. Helyettesítési szabályok

A helyettesítési szabályok az elsőrendű logikában azokat a műveleteket szabályozzák, amelyek során egy változót egy másik kifejezésre cserélünk. Ezek az egyenlőségek a logikai műveletek és a helyettesítés közötti interakciót biztosítják.

Egy művelet és a helyettesítés között - kategóriaelméleti szavakkal mondva - természetességi kapcsolat van. Ez azt jelenti, hogy a művelet kompatibilis a helyettesítéssel, azaz ha először végezzük el a műveletet, majd utána a helyettesítést, az ugyanazt az eredményt adja, mintha először végeznénk el a helyettesítést, majd utána a műveletet. Ez biztosítja, hogy a művelet és a helyettesítés nem befolyásolják egymás működését, függetlenül az alkalmazás sorrendjétől.

Ezekre a szabályokra tekinthetünk a helyettesítés implementációjaként is, legyen az egy rekurzív függvény, amely az operátorok szerinti rekurzióval van megadva, vagy mintaillesztés egy adattípusra. Azonban ezekre a megadásokra ebben az esetben nincs lehetőségünk, mivel egy record adatstruktúrát fogalmazunk meg.

A további műveleteknél is lesznek ilyen helyettesítési szabályok, ott már ezt nem részletezzük.

5.7. Logikai összekötők

5.7.1. Implikáció ($\supset$)

Az $A \supset B$ jelentése: "ha A igaz, akkor B is igaz".

$$_ \supset _ : \forall \{\Gamma\} \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma$$

Az implikáció helyettesítési szabálya:

$$\supset [] : \forall \{\Gamma \ A \ B \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow (A \supset B) \ [\gamma]^f \equiv A \ [\gamma]^f \supset B \ [\gamma]^f$$

- Az implikáció bevezető szabálya ($\supset \text{in}$): Ha tudjuk bizonyítani, hogy B igaz egy olyan kontextusban, ahol A feltételezett, akkor van egy bizonyításunk $A \supset B$ -re is.

$$\supset \text{in} : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } (\Gamma \ \text{p}_p \ A) \ (B \ [\text{p}_p]^f) \rightarrow \text{Pf } \Gamma \ ((A \supset B))$$

- Az implikáció kivezető szabálya ($\supset \text{out}$): Ha van egy bizonyításunk $A \supset B$ -re, valamint A -ra is, akkor van egy bizonyításunk B -re is.

$$\supset \text{out} : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma \ (A \supset B) \rightarrow \text{Pf } \Gamma \ A \rightarrow \text{Pf } \Gamma \ B$$

5.7.2. Konjunkció ($\wedge$)

Az $A \wedge B$ azt jelenti, hogy mind A , mind B igaz.

$$_ \wedge _ : \forall \{\Gamma\} \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma$$

Az konjunkció helyettesítési szabálya:

$$\wedge [] : \forall \{\Gamma \ A \ B \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow (A \wedge B) \ [\gamma]^f \equiv A \ [\gamma]^f \wedge B \ [\gamma]^f$$

- A konjunkció bevezető szabálya ($\wedge \text{in}$): Ha A és B külön-külön bizonyítható, akkor $A \wedge B$ is igaz.

$$\wedge \text{in} : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma \ A \rightarrow \text{Pf } \Gamma \ B \rightarrow \text{Pf } \Gamma \ (A \wedge B)$$

- Az konjunkció kivezető szabálya ($\wedge \text{out}_1$): Az $A \wedge B$ -ből következik A .
- Az konjunkció kivezető szabálya ($\wedge \text{out}_2$): Az $A \wedge B$ -ből következik B .

$$\wedge \text{out}_1 : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma \ (A \wedge B) \rightarrow \text{Pf } \Gamma \ A$$

$$\wedge \text{out}_2 : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma \ (A \wedge B) \rightarrow \text{Pf } \Gamma \ B$$

5.7.3. Diszjunkció ($\vee$)

Az $A \vee B$ jelentése, hogy legalább az egyik (A vagy B) igaz.

$$_ \vee _ : \forall \{\Gamma\} \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma$$

Az diszjunkció helyettesítési szabálya:

$$\vee [] : \forall \{\Gamma \ A \ B \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow (A \vee B) \ [\gamma]^f \equiv A \ [\gamma]^f \vee B \ [\gamma]^f$$

- A diszjunkció bevezető szabálya ($\vee \text{in}_1$): Ha A igaz, akkor $A \vee B$ is igaz.
- A diszjunkció bevezető szabálya ($\vee \text{in}_2$): Ha B igaz, akkor $A \vee B$ is igaz.

$$\vee \text{in}_1 : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma \ A \rightarrow \text{Pf } \Gamma \ (A \vee B)$$

$$\vee \text{in}_2 : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma \ B \rightarrow \text{Pf } \Gamma \ (A \vee B)$$

- A diszjunkció kivezető szabálya ($\vee \text{out}$): Ha tudjuk, hogy $A \vee B$ igaz, és abból is következik, ha A igaz, de abból is, ha B igaz, akkor C igaz.

$$\begin{aligned} \vee \text{out} : \forall \{\Gamma \ A \ B \ C\} \rightarrow \text{Pf } (\Gamma \triangleright_p A) \ (C \ [\text{p}_p]^f) \rightarrow \text{Pf } (\Gamma \triangleright_p B) \ (C \ [\text{p}_p]^f) \\ \rightarrow \text{Pf } \Gamma \ (A \vee B) \rightarrow \text{Pf } \Gamma \ C \end{aligned}$$

5.7.4. Konstans igaz ($\top$)

A $\top$ egy mindig igaz állítást jelöl.

$$\top : \forall \{\Gamma\} \rightarrow \text{For } \Gamma$$

A $\top$ helyettesítési szabálya:

$$\top [] : \forall \{\Gamma \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow \top \ [\gamma]^f \equiv \top$$

- Az *igaz* bevezető szabálya ($\top \text{in}$): Minden kontextusban igaz.

$$\top \text{in} : \forall \{\Gamma\} \rightarrow \text{Pf } \Gamma \ \top$$

5.7.5. Konstans hamis ($\perp$)

Az $\perp$ egy ellentmondást vagy hamis állítást jelöl.

$$\perp : \forall \{\Gamma\} \rightarrow \text{For } \Gamma$$

A $\perp$ helyettesítési szabálya:

$$\perp[] : \forall \{\Gamma \Delta\} \{\gamma : \text{Sub } \Delta \Gamma\} \rightarrow \perp [\gamma]^f \equiv \perp$$

- A *hamis* kivezető szabálya ($\perp\text{out}$): Ha egy bizonyítás során elérünk egy ellentmondást ($\perp$), akkor bármilyen tetszőleges állítást igaznak vehetünk (*ex falso quodlibet*).

$$\perp\text{out} : \forall \{\Gamma A\} \rightarrow \text{Pf } \Gamma \perp \rightarrow \text{Pf } \Gamma A$$

5.7.6. Kvantorok

A kvantorok formális logikai eszközök, amelyek segítségével általános és létezési állításokat tehetünk. Bindernek (kötés) is nevezzük őket, utalva rá, hogy egy változót köt, azaz meghatározza, hogy egy változó miként értelmezhető egy adott hatókörben.

Univerzális kvantor ($\forall$)

Az univerzális kvantor segítségével tudjuk kifejezni, hogy egy állítás minden lehetséges elemre igaz. Például a $\forall x P(x)$ azt jelenti, hogy minden x -re igaz, hogy $P(x)$. A deklarációban látható is, hogy mivel egy kötésről beszélünk, egy formulában lévő szabad változót kötünk meg, így egy szűkebb kontextusban lévő formulát fogunk kapni.

$$\text{Forall} : \forall \{\Gamma\} \rightarrow \text{For } (\Gamma \triangleright_t) \rightarrow \text{For } \Gamma$$

Az univerzális kvantor helyettesítési szabálya:

$$\begin{aligned} \text{Forall}[] : \forall \{\Gamma A \Delta\} \{\gamma : \text{Sub } \Delta \Gamma\} \\ \rightarrow \text{Forall } A [\gamma]^f \equiv \text{Forall } (A [\gamma \circ p_t \text{ , } t \text{ } q_t]^f) \end{aligned}$$

- Az univerzális kvantor bevezető szabálya ($\forall\text{in}$):

$$\forall\text{in} : \forall \{\Gamma A\} \rightarrow \text{Pf } (\Gamma \triangleright_t) A \rightarrow \text{Pf } \Gamma (\text{Forall } A)$$

- Az univerzális kvantor kivezető szabálya ($\forall\text{out}$):

$$\forall\text{out} : \forall\{\Gamma \vdash A\} \rightarrow \text{Pf } \Gamma \text{ (Forall } A) \rightarrow \text{Pf } (\Gamma \triangleright_t) A$$

Egzisztenciális kvantor ($\exists$)

Az egzisztenciális kvantor azt fejezi ki, hogy létezik legalább egy olyan elem, amelyre igaz egy állítás. Például a $\exists x P(x)$ azt jelenti, hogy létezik legalább egy x , amelyre igaz, hogy $P(x)$.

$$\exists : \forall\{\Gamma\} \rightarrow \text{For } (\Gamma \triangleright_t) \rightarrow \text{For } \Gamma$$

Az egzisztenciális kvantor helyettesítési szabálya:

$$\exists[] : \forall\{\Gamma \vdash A \Delta\}\{\gamma : \text{Sub } \Delta \Gamma\} \rightarrow \exists A [\gamma]^f \equiv \exists (A [\gamma \circ p_t, t q_t]^f)$$

- Az egzisztenciális kvantor bevezető szabálya ($\exists\text{in}$):

$$\exists\text{in} : \forall\{\Gamma \vdash A\} \rightarrow (t : \text{Tm } \Gamma) \rightarrow \text{Pf } \Gamma (A [\text{id}, t]^f) \rightarrow \text{Pf } \Gamma (\exists A)$$

- Az egzisztenciális kvantor kivezető szabálya ($\exists\text{out}$):

$$\exists\text{out} : \forall\{\Gamma \vdash A \vdash C\} \rightarrow \text{Pf } (\Gamma \triangleright_t \triangleright_p A) (C [p_t \circ p_p]^f) \rightarrow \text{Pf } \Gamma (\exists A) \rightarrow \text{Pf } \Gamma C$$

6. fejezet

Szintaxis

Ebben a fejezetben az elsőrendű logika szintaxisának megadását ismertetjük. Definiáljuk a szükséges szortokat és azok konstruktorait, valamint bemutatjuk, hogyan építjük fel a termek, formulák és bizonyítások struktúráját induktív adattípusokkal, ezáltal megadva szintaxist egyenlőségek, azaz kvóciensek nélkül.

A szintaxis az egyik specifikus algebrája az algebrai struktúrának, más néven iniciális algebrának is nevezzük. Az összes többi algebrát szemantikának nevezzük, erre láthatunk majd példát a következő fejezetben.

Az elsőrendű logika iniciális modelljének implementációját algebrai adattípusokkal adjuk meg, miközben biztosítjuk, hogy az egyenlőségek is teljesüljenek. Ez hasonló megközelítést igényel, mint amit a bevezetésben a monoidok esetében láttunk, ahol a listákat használtuk reprezentációként, így kvóciensek nélkül megadva a szintaxist.

6.1. Az implementáció alapelvei

A formulák létrehozó operátorait a **For** adattípus konstruktorai adják meg, kivéve a helyettesítést, amit rekurzívan definiálunk. A termek esetében a konstruktorokat a **Tm** adattípus tartalmazza, és itt is rekurzívan adjuk meg a helyettesítést. A bizonyítások hasonlóan a **Pf** adattípussal vannak megadva, azonban ebben az esetben a helyettesítés egyenlőségei is konstruktorok lesznek, amire lentebb láthatunk részletesebb magyarázatot.

6.1.1. A helyettesítések kezelése

A helyettesítések kezelésére a következő megközelítést alkalmazzuk:

- A helyettesítést rekurzívan definiáljuk a termek és formulák esetében.
- Többlépcsős megközelítés: Először a változókra definiáljuk a helyettesítést, majd ezt kiterjesztjük az összes formulára és termre

Ez a technika teszi lehetővé, hogy kvóciensek nélkül tudjuk megadni a szintaxist.

A bizonyítások esetében már nincs szükség rekurzív helyettesítésre, mivel ezek a **Prop** típusban vannak, nem pedig **Set**-ben. A kettő között az a különbség, hogy az utóbbi *proof-irrelevant*, ami röviden azt jelenti, hogy egy **Prop**-ban lévő típusnak minden eleme definicionálisan egyenlőnek tekinthető. Itt tehát elegendő a helyettesítési egyenlőségeket egyszerű konstruktorként megadni a **Prop** típus sajátosságai miatt, mivel minden egyenlőség triviálisan igaz lesz.

Összefoglalva, az iniciális modell algebrai adattípusokkal van megadva, az egyenlőségekhez pedig rekurziót használunk, kivéve a bizonyítások esetében.

6.2. Szortok

A szortokat algebrai adattípusokkal definiáljuk:

```
data Con : Set
data Sub : Con → Con → Set
data For : Con → Set
data Tm : Con → Set
```

A konstruktorokat megadjuk intuitívan a **Con** és **For** szortokhoz:

```
data Con where
  ◇      : Con
  _>_t   : Con → Con

data For where
  _⊃_    : ∀{Γ} → For Γ → For Γ → For Γ
  _∧_    : ∀{Γ} → For Γ → For Γ → For Γ
  ⊤      : ∀{Γ} → For Γ
```

$$\begin{aligned}
V &: \forall\{\Gamma\} \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma \\
\perp &: \forall\{\Gamma\} \rightarrow \text{For } \Gamma \\
\text{Forall} &: \forall\{\Gamma\} \rightarrow \text{For } (\Gamma \triangleright_t) \rightarrow \text{For } \Gamma \\
\exists &: \forall\{\Gamma\} \rightarrow \text{For } (\Gamma \triangleright_t) \rightarrow \text{For } \Gamma \\
\text{Rel} &: \forall\{\Gamma\}\{n : \mathbb{N}\} \rightarrow \text{relat } n \rightarrow \text{Tm } \Gamma \wedge n \rightarrow \text{For } \Gamma
\end{aligned}$$

6.3. Változók

Korábban már szó volt róla, hogy a modellfogalomban egységesen kezeljük a term- és bizonyítás-változókat, itt viszont szükség lesz arra, hogy a hozzájuk tartozó kontextust és helyettesítést külön valósítsuk meg. Így lesz majd egy termkontextusunk és egy bizonyítás-kontextusunk, valamint egy termhelyettesítésünk és egy bizonyításhelyettesítésünk is.

A nehézség csak az lesz, hogy ez a két rész nem független egymástól, ugyanis egy bizonyítás-kontextus kiterjesztése függeni fog egy formulától, a formulák viszont a termkontextustól függenek. Ez azt jelenti tehát, hogy a bizonyítások kontextusa függeni fog a termék kontextusától.

6.4. Termváltozók

Mivel a nyelvünk változókat is tartalmazhat, ezért be kell vezetnünk két további adattípust, a termváltozókat és a hozzá tartozó behelyettesítéseket:

$$\begin{aligned}
&\text{data VTm} : \text{Con} \rightarrow \text{Set where} \\
&\quad \text{vz} : \forall\{\Gamma\} \rightarrow \text{VTm } (\Gamma \triangleright_t) \\
&\quad \text{vs} : \forall\{\Gamma\} \rightarrow \text{VTm } \Gamma \rightarrow \text{VTm } (\Gamma \triangleright_t) \\
&\text{data VSub} : \text{Con} \rightarrow \text{Con} \rightarrow \text{Set where} \\
&\quad \varepsilon : \forall\{\Gamma\} \rightarrow \text{VSub } \Gamma \diamond \\
&\quad _.\text{v}_ : \forall\{\Gamma \Delta\} \rightarrow \text{VSub } \Delta \Gamma \rightarrow \text{VTm } \Delta \rightarrow \text{VSub } \Delta (\Gamma \triangleright_t)
\end{aligned}$$

6.5. Termek és behelyettesítések

Az előbbieket segítségével már meg tudjuk adni a **Tm** és **Sub** szortokat is:

data Tm where

var : $\forall\{\Gamma\} \rightarrow \text{VTm } \Gamma \rightarrow \text{Tm } \Gamma$
 fun : $\forall\{\Gamma\}\{n : \mathbb{N}\} \rightarrow \text{funar } n \rightarrow \text{Tm } \Gamma \wedge n \rightarrow \text{Tm } \Gamma$

data Sub where

ε : $\forall\{\Gamma\} \rightarrow \text{Sub } \Gamma \diamond$
 $_.\text{t}_$: $\forall\{\Gamma \Delta\} \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{Tm } \Delta \rightarrow \text{Sub } \Delta (\Gamma \triangleright_{\text{t}})$

6.6. Bizonyítások kontextusa és azok behelyettesítése

A bizonyítások miatt azonban szükségünk van még egy összetevőre, ez lesz a bizonyítások kontextusa:

data Conp : Con $\rightarrow$ Set where

$\diamond_{\text{p}}$: $\forall\{\Gamma\} \rightarrow \text{Conp } \Gamma$
 $_.\triangleright_{\text{p}}_$: $\forall\{\Gamma\} \rightarrow \text{Conp } \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{Conp } \Gamma$

Továbbá szükségünk lesz még a Conp behelyettesítésére is, hogy a bizonyítások szortját meg tudjuk adni:

data Pf : $(\Gamma : \text{Con}) (\Gamma_{\text{p}} : \text{Conp } \Gamma) \rightarrow \text{For } \Gamma \rightarrow \text{Prop}$

data Subp : $(\Gamma : \text{Con}) \rightarrow \text{Conp } \Gamma \rightarrow \text{Conp } \Gamma \rightarrow \text{Prop}$ where

idp : $\forall\{\Gamma \Gamma_{\text{p}}\} \rightarrow \text{Subp } \Gamma \Gamma_{\text{p}} \Gamma_{\text{p}}$
 ε : $\forall\{\Gamma \Gamma_{\text{p}}\} \rightarrow \text{Subp } \Gamma \Gamma_{\text{p}} \diamond_{\text{p}}$
 $_.\triangleright_{\text{p}}_$: $\forall\{\Delta \Gamma_{\text{p}} \Delta_{\text{p}} A\} \rightarrow \text{Subp } \Delta \Delta_{\text{p}} \Gamma_{\text{p}} \rightarrow \text{Pf } \Delta \Delta_{\text{p}} A \rightarrow \text{Subp } \Delta \Delta_{\text{p}} (\Gamma_{\text{p}} \triangleright_{\text{p}} A)$
 p_{p} : $\forall\{\Gamma \Gamma_{\text{p}} A\} \rightarrow \text{Subp } \Gamma (\Gamma_{\text{p}} \triangleright_{\text{p}} A) (\Gamma_{\text{p}} [\text{id}_{\text{t}}] \text{Conp})$
 $_.\circ_{\text{p}}_$: $\forall\{\Xi \Theta_{\text{p}} \Delta_{\text{p}} \Gamma_{\text{p}}\} \rightarrow \text{Subp } \Xi \Delta_{\text{p}} \Gamma_{\text{p}} \rightarrow \text{Subp } \Xi \Theta_{\text{p}} \Delta_{\text{p}} \rightarrow \text{Subp } \Xi \Theta_{\text{p}} \Gamma_{\text{p}}$
 $_.[_]\text{p}$: $\forall\{\Xi \Psi \Delta_{\text{p}} \Gamma_{\text{p}}\} \rightarrow \text{Subp } \Xi \Delta_{\text{p}} \Gamma_{\text{p}} \rightarrow (\xi : \text{Sub } \Psi \Xi) \rightarrow \text{Subp } \Psi (\Delta_{\text{p}} [\xi] \text{Conp}) (\Gamma_{\text{p}} [\xi] \text{Conp})$

6.7. Bizonyítások

data Pf where

$_.[_]\text{p}$: $\forall\{\Gamma \Gamma_{\text{p}} \Delta A\} \rightarrow \text{Pf } \Gamma \Gamma_{\text{p}} A \rightarrow (\gamma : \text{Sub } \Delta \Gamma)$

$$\begin{aligned}
& \rightarrow \text{Pf } \Delta (\Gamma_p [\gamma] \text{Comp}) (A [\gamma]^f) \\
__[_]^{pp} & : \forall \{ \Xi \Gamma_p \Delta_p A \} \rightarrow \text{Pf } \Xi \Gamma_p A \rightarrow (\gamma_p : \text{Subp } \Xi \Delta_p \Gamma_p) \rightarrow \text{Pf } \Xi \Delta_p A \\
q_p & : \forall \{ \Gamma \Gamma_p A \} \rightarrow \text{Pf } \Gamma (\Gamma_p \triangleright_p A) A \\
\supset_{\text{in}} & : \forall \{ \Gamma \Gamma_p A B \} \rightarrow \text{Pf } \Gamma (\Gamma_p \triangleright_p A) B \rightarrow \text{Pf } \Gamma \Gamma_p (A \supset B) \\
\supset_{\text{out}} & : \forall \{ \Gamma \Gamma_p A B \} \rightarrow \text{Pf } \Gamma \Gamma_p (A \supset B) \rightarrow \text{Pf } \Gamma \Gamma_p A \rightarrow \text{Pf } \Gamma \Gamma_p B \\
\wedge_{\text{in}} & : \forall \{ \Gamma \Gamma_p A B \} \rightarrow \text{Pf } \Gamma \Gamma_p A \rightarrow \text{Pf } \Gamma \Gamma_p B \rightarrow \text{Pf } \Gamma \Gamma_p (A \wedge B) \\
\wedge_{\text{out}_1} & : \forall \{ \Gamma \Gamma_p A B \} \rightarrow \text{Pf } \Gamma \Gamma_p (A \wedge B) \rightarrow \text{Pf } \Gamma \Gamma_p A \\
\wedge_{\text{out}_2} & : \forall \{ \Gamma \Gamma_p A B \} \rightarrow \text{Pf } \Gamma \Gamma_p (A \wedge B) \rightarrow \text{Pf } \Gamma \Gamma_p B \\
\top_{\text{in}} & : \forall \{ \Gamma \Gamma_p \} \rightarrow \text{Pf } \Gamma \Gamma_p \top \\
\vee_{\text{in}_1} & : \forall \{ \Gamma \Gamma_p A B \} \rightarrow \text{Pf } \Gamma \Gamma_p A \rightarrow \text{Pf } \Gamma \Gamma_p (A \vee B) \\
\vee_{\text{in}_2} & : \forall \{ \Gamma \Gamma_p A B \} \rightarrow \text{Pf } \Gamma \Gamma_p B \rightarrow \text{Pf } \Gamma \Gamma_p (A \vee B) \\
\vee_{\text{out}} & : \forall \{ \Gamma \Gamma_p A B C \} \rightarrow \text{Pf } \Gamma (\Gamma_p \triangleright_p A) (C) \rightarrow \text{Pf } \Gamma (\Gamma_p \triangleright_p B) (C) \\
& \rightarrow \text{Pf } \Gamma \Gamma_p (A \vee B) \rightarrow \text{Pf } \Gamma \Gamma_p C \\
\perp_{\text{out}} & : \forall \{ \Gamma \Gamma_p A \} \rightarrow \text{Pf } \Gamma \Gamma_p \perp \rightarrow \text{Pf } \Gamma \Gamma_p A \\
\forall_{\text{in}} & : \forall \{ \Gamma \Gamma_p A \} \rightarrow \text{Pf } (\Gamma \triangleright_t) (\Gamma_p [\rho_t] \text{Comp}) A \rightarrow \text{Pf } \Gamma \Gamma_p (\text{Forall } A) \\
\forall_{\text{out}} & : \forall \{ \Gamma \Gamma_p A \} \rightarrow \text{Pf } \Gamma \Gamma_p (\text{Forall } A) \rightarrow \text{Pf } (\Gamma \triangleright_t) (\Gamma_p [\rho_t] \text{Comp}) A \\
\exists_{\text{in}} & : \forall \{ \Gamma \Gamma_p A \} \rightarrow (t : \text{Tm } \Gamma) \rightarrow \text{Pf } \Gamma \Gamma_p (A [\text{id}_t, t]^f) \rightarrow \text{Pf } \Gamma \Gamma_p (\exists A) \\
\exists_{\text{out}} & : \forall \{ \Gamma \Gamma_p A C \} \rightarrow \text{Pf } (\Gamma \triangleright_t) (\Gamma_p [\rho_t] \text{Comp} \triangleright_p A) (C [\rho_t]^f) \\
& \rightarrow \text{Pf } \Gamma \Gamma_p (\exists A) \rightarrow \text{Pf } \Gamma \Gamma_p C
\end{aligned}$$

6.8. Inicialis modell

Az inicialis modell tehát a következőképpen néz ki:

$$\begin{aligned}
I & : \text{Model} \\
I & = \text{record} \\
\{ \text{Con} & = \Sigma \text{Con Comp} \\
; \text{Sub} & = \lambda (\Delta, \Delta_p) (\Gamma, \Gamma_p) \\
& \rightarrow \Sigma (\text{Sub } \Delta \Gamma) \lambda \gamma \rightarrow \text{Lift } (\text{Subp } \Delta \Delta_p (\Gamma_p [\gamma] \text{Comp})) \\
; _\circ_ & = \lambda \{ (\Gamma, \Gamma_p) (\Delta, \Delta_p) (\theta, \theta_p) \} (\gamma, \text{mk } \gamma_p) (\delta, \text{mk } \delta_p) \\
& \rightarrow (\gamma \circ \delta), \text{mk } (\text{substP } (\text{Subp } \theta \theta_p) ([\circ] \text{Comp}^{-1}) ((\gamma_p [\delta]^p) \circ_p \delta_p)) \\
; \text{ass} & = \text{cong, ass} \\
; \text{id} & = \lambda \{ (\Gamma, \Gamma_p) \} \rightarrow \text{id}_t, \text{mk } (\text{substP } (\lambda x \rightarrow \text{Subp } \Gamma \Gamma_p x) ([\text{id}] \text{Comp}^{-1}) \text{idp}) \\
; \text{idl} & = \text{cong, idl}
\end{aligned}$$

```

; idr = cong, idr
; ◇ = ◇ , ◇p
; ε = ε , mk ε
; ◇η = cong (◇, mk ◇) ◇η
; Tm = λ (Γ , Γp) → Tm Γ
; [ ]t = λ t (s , sp) → t [ s ]t
; [o]t = λ {Γ Δ Θ t (γ , γp) (δ , δp)} → [o]t {t = t}
; [id]t = t[id]t
; [ ]▷t = λ (Γ , Γp) → Γ ▷t , Γp [ pt ]Conp
; [ ]t,▷ = λ (γ , mk γp) t
    → (γ ,t t) ,
    mk (substP (Subp [ ]t,▷) (cong (◇ [ ]t,▷) (▷tβ1-1) □ [o]t,▷) γp)
; pt = pt , mk idp
; qt = qt
; ▷tβ1 = cong, ▷tβ1
; ▷tβ2 = refl
; ▷tη = cong, ▷tη
; For = λ (Γ , Γp) → For Γ
; [ ]f = λ A (s , sp) → A [ s ]f
; [o]f = [o]f
; [id]f = f[id]f
; Pf = λ (Γ , Γp) A → Pf Γ Γp A
; [ ]p = λ { {Γ , Γp} {Δ , Δp} {A} } a (γ , mk γp) → (a [ γ ]p) [ γp ]pp }
; [ ]▷p = λ (Γ , Γp) A → Γ , Γp ▷p A
; [ ]▷,p = λ (s , (mk sp)) p → s , mk (sp ,p p)
; pp = idt , mk pp
; qp = λ { (Γ , Γp) } {A} → substP (λ x → Pf Γ (Γp ▷p A) x) (f[id]f-1) qp
; ▷pβ1 = cong, idl
; ▷pη = cong, idl-1
; Rel = Rel
; Rel[] = λ {Γ n ar} → cong (λ x → Rel ar x) s≡map
; fun = fun
; fun[] = λ {Γ n ar} → cong (λ x → fun ar x) s≡map
; [ ]▷ = [ ]▷

```

```

;  $\supset [] = \text{refl}$ 
;  $\supset \text{in} = \lambda \{(\Gamma, \Gamma_p) A B\} \rightarrow \text{substP } (\lambda x \rightarrow \text{Pf } \Gamma (\Gamma_p \triangleright_p A) x$ 
     $\rightarrow \text{Pf } \Gamma \Gamma_p (A \supset B)) (\text{f[id]}^f)^{-1} (\supset \text{in})$ 
;  $\supset \text{out} = \supset \text{out}$ 
;  $\_ \wedge \_ = \_ \wedge \_$ 
;  $\wedge [] = \text{refl}$ 
;  $\wedge \text{in} = \wedge \text{in}$ 
;  $\wedge \text{out}_1 = \wedge \text{out}_1$ 
;  $\wedge \text{out}_2 = \wedge \text{out}_2$ 
;  $\top = \top$ 
;  $\top [] = \text{refl}$ 
;  $\top \text{in} = \top \text{in}$ 
;  $\_ \vee \_ = \_ \vee \_$ 
;  $\vee [] = \text{refl}$ 
;  $\vee \text{in}_1 = \vee \text{in}_1$ 
;  $\vee \text{in}_2 = \vee \text{in}_2$ 
;  $\vee \text{out} = \lambda \{(\Gamma, \Gamma_p)\} \{A\} \{B\} \{C\} a b$ 
     $\rightarrow \text{substP } (\lambda C \rightarrow \text{Pf } \Gamma \Gamma_p (A \vee B) \rightarrow \text{Pf } \Gamma \Gamma_p C)$ 
     $\text{f[id]}^f (\vee \text{out } a b)$ 
;  $\perp = \perp$ 
;  $\perp [] = \text{refl}$ 
;  $\perp \text{out} = \perp \text{out}$ 
;  $\text{Forall} = \text{Forall}$ 
;  $\text{Forall} [] = \text{cong } (\lambda z \rightarrow \text{Forall } (\_ [z, \text{t} \text{ var } \text{vz}]^f)) (\text{op}^{-1})$ 
;  $\forall \text{in} = \forall \text{in}$ 
;  $\forall \text{out} = \forall \text{out}$ 
;  $\exists = \exists$ 
;  $\exists [] = \text{cong } (\lambda z \rightarrow \exists (\_ [z, \text{t} \text{ var } \text{vz}]^f)) (\text{op}^{-1})$ 
;  $\exists \text{in} = \exists \text{in}$ 
;  $\exists \text{out} = \lambda \{(\Gamma, \Gamma_p)\} \{A\} \{C\} w p$ 
     $\rightarrow \exists \text{out } \{\Gamma\} \{\Gamma_p\} \{A\} \{C\}$ 
     $(\text{substP } (\lambda z$ 
     $\rightarrow \text{Pf } (\Gamma \triangleright_t) ((\Gamma_p [p_t] \text{Conp}) \triangleright_p A) (C [z]^f)) \text{idr } w) p$ 
     $\}$ 

```

Láthatjuk, hogy a kontextus és a helyettesítés a korábban is említett különválasztott term- és bizonyítás-kontextus valamint a term- és bizonyítás-helyettesítés párok fogják megadni, ebből adódik a további részek kicsivel összetettebb megadása.

7. fejezet

Modellek

Ebben a fejezetben különböző szemantikai modelleket definiálunk az elsőrendű logikához. Bemutatjuk a Bool, a Tarski és a Family modelleket, részletezve azok felépítését, működését és alkalmazhatóságát a formalizált logikai rendszeren belül.

7.1. Szemantikai modellek

A szemantikai modellek a formalizált elsőrendű logika jelentését, értelmezését adják meg. A három vizsgált modell különböző megközelítéseket kínál a logikai kifejezések értelmezésére és kiértékelésére.

- Bool modell: A legegyszerűbb és legközvetlenebb megközelítés, amely a logikai kifejezéseket az igaz és hamis értékekre képezi le, valamint a logikai műveletek a Bool értékek közötti műveletekkel azonosak. Ez a klasszikus kétértékű szemantika.
- Tarski modell: A logikai kifejezések értelmezését a halmazelmélet alapjaira helyezi. Ebben a modellben a logikai változók és kifejezések értékei halmazok, illetve halmazok közötti relációk. A Tarski szemantika a logikai igazságot a kifejezések halmazelméleti interpretációjával határozza meg, amely lehetővé teszi a logikai állítások formális és precíz vizsgálatát.
- Family modell: Egy indexelt családok segítségével felépített szemantika. Felfogható a Kripke-szemantika egy specializációjaként. Ez a modell, a Kripke modellel szemben, nem specifikál explicit elérhetőségi relációt az indexek között, ami egyszerűsíti a struktúrát.

7.2. Bool modell

A Bool modellhez közvetlenül az Agda beépített Bool típusát használjuk, így a logikai összekötők ($\wedge$, $\vee$, $\rightarrow$, $\neg$) is a megfelelő Bool műveletekre fordítódnak. A formulák kiértékelése során minden atomi állítás, predikátum és függvény végső soron egy Bool értékre redukálódik. Az univerzális és egzisztenciális kvantorokra gondolhatunk úgy, mint listák feletti `all` és `any` műveletek.

A Bool modell csak a klasszikus metaelméletben működik, ahol teljes a klasszikus logikára – a teljesség azt jelenti, hogy ha egy állítás teljesül (tehát `=true`) a Bool-modellben, akkor a szintaxisban van bizonyítása ugyanannak az állításnak.

A modell megadásához a következő paraméterekre lesz szükségünk:

```
(funar : ℕ → Set)
(relar : ℕ → Set)
(D : Set)
(fun : (n : ℕ) → funar n → D → n → D)
(Rel : (n : ℕ) → relar n → D → n → Bool)
```

Paraméter	Jelentés
<code>funar</code>	A függvény szimbólumok halmaza aritás szerint
<code>relar</code>	A relációs szimbólumok halmaza aritás szerint
<code>D</code>	Az alap halmaz (a modell tartománya)
<code>fun</code>	A függvény szimbólumok értelmezése (szemantikai jelentés)
<code>Rel</code>	A relációs szimbólumok értelmezése (szemantikai jelentés)

7.1. táblázat. Bool modell modulparaméterek

Ezt a modellt viszonylag egyszerű implementálni, azonban a klasszikus metaelmélet miatt szükségünk lesz a kizárt harmadik elvére, amit meg kell mondanunk az Agdának is hogy létezik, hogy meg tudjuk adni a kvantorok implementációját.

postulate

```
lem : (A : Prop) → A ⊔ (A → 0p)
```

Így a kvantorok implementációja a következőképpen néz ki:

```
Forall : {Γ : Set} → (Γ × D → Bool) → Γ → Bool
Forall F γ with lem ((d : D) → F(γ , d) ≡ true)
```

```
... | inl x = true
... | inr x = false
```

```
∃ : {Γ : Set} → (Γ × D → Bool) → Γ → Bool
∃ A γ with lem ((Σsp D λ d → A (γ , d) ≡ true))
... | inl x = true
... | inr x = false
```

A többi logikai operátor és szabály implementációja magától értetődő, és az Agdában egyszerűen mintaillesztéssel van megadva. Az operátorok definíciói követik a Bool típus logikai műveleteit, így az implementációk rövidek és könnyen érthetőek.

```
B : Model funar relar
B = record
  { Con = Set
  ; Sub = λ Δ Γ → Δ → Γ
  ; Tm = λ Γ → Γ → D
  ; For = λ Γ → Γ → Bool
  ; Pf = λ Γ A → (γ : Γ) → A γ ≡ true
  ; _⊃_ = λ A B γ → A γ => B γ
  ; _∧_ = λ A B γ → A γ && B γ
  ; _∨_ = λ A B γ → A γ || B γ
  ; Forall = Forall
  ; ∃ = ∃
  ;
  }
```

A kódban látott `=>`, `&&` és `||` operátorok a Bool típusra értelmezett logikai operátorok, amelyek értelemszerűen az implikációt, a konjunkciót és a diszjunkciót valósítják meg.

7.3. Tarski modell

A Tarski modell az elsőrendű logika standard szemantikai modellje, de nem teljes; nincs szükség a kizárt harmadik elvére a megadásához. Egy nem üres domain (univerzum) felett értelmezett interpretáció.

Az Agdában implementált Tarski modell a következő komponensekből áll:

- Egy tetszőleges típus, amely a domaint reprezentálja
- Függvények, amelyek a logikai nyelv szimbólumait a domain elemeire, függvényeire és relációira képezik le
- Változókiértékelő függvény, amely a változókat a domain elemeire képezi le
- Rekurzív interpretációs függvény, amely a formulákat a domain feletti állításokra képezi le

Itt már nincs szükség a klasszikus metaelméletre; a kizárt harmadik elve csak akkor teljesül a modellben, ha a metaelméletben is teljesül.

A modell megadásához a következő paraméterekre lesz szükségünk:

$(funar : \mathbb{N} \rightarrow \text{Set})$
 $(relar : \mathbb{N} \rightarrow \text{Set})$
 $(D : \text{Set})$
 $(fun : (n : \mathbb{N}) \rightarrow funar\ n \rightarrow D \wedge n \rightarrow D)$
 $(Rel : (n : \mathbb{N}) \rightarrow relar\ n \rightarrow D \wedge n \rightarrow \text{Prop})$

Paraméter	Jelentés
funar	A függvény szimbólumok halmaza aritás szerint
relar	A relációs szimbólumok halmaza aritás szerint
D	Az alap halmaz (a modell tartománya)
fun	A függvény szimbólumok értelmezése (szemantikai jelentés)
Rel	A relációs szimbólumok értelmezése (szemantikai jelentés)

7.2. táblázat. Tarski modell modulparaméterek

A modell implementációja így néz ki:

$T : \text{Model } funar\ relar$
 $T = \text{record}$
 $\{ \text{Con} = \text{Set}$
 $; \text{Sub} = \lambda \Delta \Gamma \rightarrow \Delta \rightarrow \Gamma$

```

; Tm =  $\lambda \Gamma \rightarrow \Gamma \rightarrow D$ 
; For =  $\lambda \Gamma \rightarrow \Gamma \rightarrow \mathbf{Prop}$ 
; Pf =  $\lambda \Gamma A \rightarrow (\gamma : \Gamma) \rightarrow A \gamma$ 
; ⊃ =  $\lambda A B \gamma \rightarrow A \gamma \rightarrow B \gamma$ 
; ∧ =  $\lambda A B \gamma \rightarrow A \gamma \times_{\mathbf{p}} B \gamma$ 
; ∨ =  $\lambda A B \gamma \rightarrow A \gamma \uplus_{\mathbf{p}} B \gamma$ 
; Forall =  $\lambda A \gamma \rightarrow (d : D) \rightarrow A (\gamma, d)$ 
; ∃ =  $\lambda A \gamma \rightarrow \Sigma_{\mathbf{sp}} D \lambda d \rightarrow A (\gamma, d)$ 
;
}

```

7.4. Family modell

A Kripke modell teljes, de túl bonyolult, ezért csak egy speciális esetét, a Family modellt adjuk meg.

- Indexelés mint lehetséges világok: A Family modellben szereplő I indexhalmaz elemei megfeleltethetők a Kripke szemantika lehetséges világainak.
- Az elérhetőségi reláció implicit módon adott vagy triviális.
- A $D(i)$ tartományok explicit módon specifikáltak minden i indexre.
- A logikai operátorok szemantikája hasonlóan van definiálva, de a modell szerkezete egyszerűbb és célorientáltabb.

Ahogy a Kripke-modellben minden lehetséges világban kiértékeljük a formulákat, a Family modellben minden indexhez tartozó struktúrán értelmezzük a kifejezéseket és formulákat. Elérhetőségi reláció hiánya: A Kripke-modellekben alapvető szerepet játszik a világok közötti elérhetőségi reláció (R), amely meghatározza a modális operátorok szemantikáját. A Family modell nem specifikál explicit elérhetőségi relációt az indexek között, ami egyszerűsíti a struktúrát.

Minden predikátum egy típuscsalád, amely a domain elemeihez típusokat rendel. A logikai konnektívák típuskonstruktorokként lesznek implementálva (pl. függő összeg és szorzat típusok). A kvantorok függő típusokként jelennek meg (Π és Σ típusok). A bizonyítások a megfelelő típusok értékei.

Ez a modell részhalmaz-modellként is értelmezhető: tehát az állítások I -nek a részhalmazai.

A modell megadásához a következő paraméterekre lesz szükségünk:

$(funar : \mathbb{N} \rightarrow \text{Set})$
 $(relar : \mathbb{N} \rightarrow \text{Set})$
 $(I : \text{Set})$
 $(D : I \rightarrow \text{Set})$
 $(fun : (n : \mathbb{N}) \rightarrow funar\ n \rightarrow (i : I) \rightarrow D\ i \wedge n \rightarrow D\ i)$
 $(Rel : (n : \mathbb{N}) \rightarrow relar\ n \rightarrow (i : I) \rightarrow D\ i \wedge n \rightarrow \text{Prop})$

Paraméter	Jelentés
funar	A függvény szimbólumok halmaza aritás szerint
relar	A relációs szimbólumok halmaza aritás szerint
I	Az index halmaz
D	Az alap halmaz (a modell tartománya)
fun	A függvény szimbólumok értelmezése (szemantikai jelentés)
Rel	A relációs szimbólumok értelmezése (szemantikai jelentés)

7.3. táblázat. Family modell modulparaméterek

Majd ezek alapján a modellünk:

$F : \text{Model } funar\ relar$
 $F = \text{record}$
 $\{ \text{Con} = I \rightarrow \text{Set}$
 $; \text{Sub} = \lambda \Delta \Gamma \rightarrow (i : I) \rightarrow \Delta\ i \rightarrow \Gamma\ i$
 $; \text{Tm} = \lambda \Gamma \rightarrow (i : I) \rightarrow \Gamma\ i \rightarrow D\ i$
 $; \text{For} = \lambda \Gamma \rightarrow (i : I) \rightarrow \Gamma\ i \rightarrow \text{Prop}$
 $; \text{Pf} = \lambda \Gamma A \rightarrow (i : I) \rightarrow (\gamma : \Gamma\ i) \rightarrow A\ i\ \gamma$
 $; _ \supset _ = \lambda A B\ i\ \gamma \rightarrow (A\ i\ \gamma) \rightarrow (B\ i\ \gamma)$
 $; _ \wedge _ = \lambda A B\ i\ \gamma \rightarrow (A\ i\ \gamma) \times_p (B\ i\ \gamma)$
 $; _ \vee _ = \lambda A B\ i\ \gamma \rightarrow (A\ i\ \gamma) \uplus_p (B\ i\ \gamma)$
 $; \text{Forall} = \lambda A\ i\ \gamma \rightarrow (d : D\ i) \rightarrow A\ i\ (\gamma, d)$
 $; \exists = \lambda A\ i\ \gamma \rightarrow \Sigma_{\text{sp}} (D\ i) \lambda d \rightarrow A\ i\ (\gamma, d)$

$\vdots$
 $\}$

8. fejezet

Bizonyítás különböző modellekben

Ebben a fejezetben példákon keresztül szemléltetjük, hogyan lehet konkrét logikai állításokat bizonyítani az egyes megadott modellekben. Egyszerűbb és összetettebb formulák esetén is megvizsgáljuk, hogyan néz ki a bizonyítás a szintaxis szintjén, illetve hogyan értelmezhetők a különböző szemantikai modellekben (Bool, Tarski, Family). Külön hangsúlyt fektetünk arra, hogy az egyes modellekben való bizonyítás mennyiben könnyebb vagy mikben tér el.

8.1. Egyszerűbb formula

8.1.1. Informális bizonyítás

8.1.2. Bizonyítás a szintaxisban

8.1.3. Bizonyítás a Bool modellben

8.1.4. Bizonyítás a Tarski modellben

8.1.5. Bizonyítás a Family modellben

8.2. Összetettebb formula

8.2.1. Informális bizonyítás

8.2.2. Bizonyítás a szintaxisban

8.2.3. Bizonyítás a Bool modellben

8.2.4. Bizonyítás a Tarski modellben

8.2.5. Bizonyítás a Family modellben

9. fejezet

Összegzés

A dolgozat célja az elsőrendű logika algebrai szemléletű formalizálása volt az Agda bizonyító asszisztens segítségével. A formalizáció során az elsőrendű logikát mélyen beágyazott nyelvként írtuk le, majd több különböző szemantikai modelltadtunk meg: a Bool modellt, a Tarski modellt, valamint a Family modellt.

A munka során sikerült egy általános, algebrai alapokon nyugvó rendszert felépíteni, amely képes kifejezni az elsőrendű logika legfontosabb elemeit, és megfelelő alapot ad további szemantikai vizsgálatokhoz. A szintaxis és a modellek kialakítása biztosítja, hogy a rendszer bővíthető és adaptálható legyen különböző alternatív szemantikai értelmezésekhez.

Ugyanakkor több fejlesztési irány is nyitva maradt, amelyek tovább növelhetik a formalizáció teljességét és gyakorlati alkalmazhatóságát:

- Kripke szemantika teljes megadása: Jelen dolgozatban a Family modell segítségével közelítettük meg a Kripke-féle szemantikát, azonban annak teljes, formálisan megadott változata és részletes leírása további munka tárgyát képezheti.
- Iterátor implementáció és inicialitás bizonyítása: Bár a dolgozatban a szintaxist iniciális modellként konstruáltuk meg, az iniciális tulajdonság teljes, formális bizonyítása — vagyis annak kimutatása, hogy bármely más modellbe egy és csak egy homomorfizmus létezik, azaz ha van egy bizonyításunk a szintaxisban, akkor azt bármelyik modelben ki lehet értékelni — még hátravan.

A dolgozat eredményeként létrejött formalizáció egy stabil és bővíthető alapot biztosít az elsőrendű logika algebrai szemléletű vizsgálatához Agdában. A kidolgo-

zott szintaxis és modellek lehetővé teszik a logikai rendszerek pontos és strukturált kezelését. A munka során megjelölt továbbfejlesztési irányok — például a Kripke szemantika részletesebb kidolgozása, az inicialitás formális bizonyítása — a rendszer további mélyítését és általános használhatóságának növelését célozzák.

Irodalomjegyzék

- [1] Róbert Fenyvesi. „Formalization of Mathematical Logic”. Dipl. Eötvös Loránd Tudományegyetem, 2021.
- [2] Samy Avrillon. *Logic as a Second-Order Generalized Algebraic Theory*. Techn. jel. Faculty of Informatics, ELTE, 2023. URL: <https://github.com/MysaaJava/m1-internship/releases/download/project-report/Avrillon-02.pdf>.
- [3] Thorsten Altenkirch, Martin Hofmann és Thomas Streicher. „Categorical reconstruction of a reduction free normalization proof”. *Category Theory and Computer Science*. Szerk. David Pitt, David E. Rydeheard és Peter Johnstone. 953. köt. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 1995, 182–199. old. ISBN: 978-3-540-44661-3. DOI: 10.1007/3-540-60164-3_27.
- [4] Steve Awodey és Andrej Bauer. *Introduction to Categorical Logic*. 2020. URL: [http://mathieu.anel.free.fr/mat/80514-814/Awodey-Bauer-catlog\(2019\).pdf](http://mathieu.anel.free.fr/mat/80514-814/Awodey-Bauer-catlog(2019).pdf).
- [5] Yannick Forster, Dominik Kirst és Dominik Wehr. „Completeness Theorems for First-Order Logic Analysed in Constructive Type Theory”. *Logical Foundations of Computer Science*. Szerk. Sergei Artemov és Anil Nerode. Cham: Springer International Publishing, 2020, 47–74. old. ISBN: 978-3-030-36755-8. DOI: 10.1007/978-3-030-36755-8_4.
- [6] Gaëtan Gilbert és Olivier Hermant. „Normalisation by Completeness with Heyting Algebras”. *Logic for Programming, Artificial Intelligence, and Reasoning*. Szerk. Martin Davis és tsai. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, 469–482. old. ISBN: 978-3-662-48899-7. DOI: 10.1007/978-3-662-48899-7_33.

- [7] Leonardo de Moura és Nikolaj Bjørner. „Z3: An Efficient SMT Solver”. *Tools and Algorithms for the Construction and Analysis of Systems*. Szerk. C. R. Ramakrishnan és Jakob Rehof. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, 337–340. old. ISBN: 978-3-540-78800-3. DOI: 10.1007/978-3-540-78800-3_24.